

 Votre recherche

[Accueil \(/\)](#)

[GNU/Linux Magazine \(/GNU-Linux-Magazine\)](#)

[GLMF-206 \(/GNU-Linux-Magazine/glmf-206\)](#)

Le test de Peter

GNU/LINUX MAGAZINE

Développement sur systèmes open source

2998 article(s)

Le test de Peter



GNU/Linux Magazine n° 206 (/GNU-Linux-Magazine/glmf-206) juillet 2017

Par Dublé Etienne (/auteur/duble-etienne)

Spécialité(s) : [Système \(/search/node?domains%5B0%5D=72451\)](/search/node?domains%5B0%5D=72451)

Tag(s) :

[Initramfs \(/search/node?tags%5B0%5D=72919\)](/search/node?tags%5B0%5D=72919)

[Hacking \(/search/node?tags%5B0%5D=72920\)](/search/node?tags%5B0%5D=72920)

On a tous un ou deux amis bizarres. C'est le cas par exemple de mon ami Peter. Dès qu'il trouve quelque chose de nouveau, il ne peut s'empêcher de faire un test. Il teste donc les spécialités culinaires, les paradigmes de programmation, les techniques de drague... Il teste absolument tout. Avant-hier, quand je l'ai aperçu à la terrasse d'un café, je me doutais donc bien qu'il était en train de tester quelque chose...



Ajouter à une liste de lecture(/readlist/get/modal/0)



Créer une alerte.



Une démo ? Un devis ?



L'IA ARRIVE SUR CONNECT :

Découvrez ED, l'assistant IA de Connect
disponible en option !

N'hésitez pas à laisser vos coordonnées professionnelles, nous vous contacterons rapidement !

ÊTES-VOUS DÉJÀ ABONNÉ À CONNECT ?

Vous disposez dès maintenant de 10 questions
gratuites pour découvrir toute la puissance de ED,
votre assistant IA !

(/advertising/redirect/141)

Coordonnées

Mardi, je suis sorti du boulot un peu moins en retard que d'habitude. En passant devant EVE, une petite cafétéria étudiante sur le campus de Grenoble, j'ai repéré Peter en train de visionner une vidéo sur son ordi portable. Il faut dire qu'il a du temps en ce moment, il s'est fait licencier récemment, suite à une histoire de produit pas assez testé (c'est en tout cas ce qu'il m'en a dit). Bref, je me suis dit que j'allais passer voir ce qu'il mijotait.

« Tiens salut » a-t-il dit en m'apercevant. « Je suis en train de tester quelque chose. Tu connais cette série US ? » a-t-il enchaîné en tournant l'écran vers moi. Je lui ai répondu que oui. Je ne suis pas un grand fan, mais ma femme aime bien. Et il m'a proposé de visionner l'extrait qui lui posait problème.

1. L'épisode

Dans cet épisode, l'agent Devon, du bureau NCIS de Chicago, est soupçonné d'être une « taupe » à la solde d'une puissance étrangère. Plutôt que de l'arrêter, Gibbs (le chef d'équipe) décide de profiter de cette situation : Devon ne sait pas qu'on le soupçonne. Si on peut trafiquer son matériel pour intercepter ses communications, on pourrait obtenir des informations importantes. Voir même le manipuler et l'entraîner vers de fausses pistes. McGee (l'expert technique) et DiNozzo (l'agent de terrain), travestis en agents d'entretien, pénètrent alors dans son bureau pendant la pause déjeuner. À ce moment-là, on voit que l'ordi de Devon est resté allumé. McGee l'arrête et le fait redémarrer sur une clé USB qu'il a sans doute préparée au préalable. On voit alors le bureau qui s'affiche puis une barre de progression, mais, déjà, l'agent Devon se dirige de nouveau vers le bâtiment. DiNozzo en sort et le croise devant la porte d'entrée. Pour gagner quelques secondes, il l'interpelle au sujet de la jolie collègue qui vient de passer. Mais l'agent Devon y prête peu d'attention et se dirige vers l'ascenseur. On voit alors la barre de progression qui se termine, McGee qui arrache aussitôt la clé USB, sort du bureau et referme la porte, juste au moment où l'agent Devon sort de l'ascenseur.

Peter positionne ensuite le curseur de la vidéo un peu plus loin, à un moment où l'agent Devon utilise à nouveau son ordi. On comprend alors que la ruse a fonctionné : il n'a visiblement remarqué aucun changement sur son système. En répondant à quelques e-mails sur son ordi trafiqué, il révèle rapidement plusieurs de ses complices.

2. « Impossible » ou juste « Invraisemblable » ?

- Tu as vu ? Tu crois que c'est possible ? me demande alors Peter.

- De réinstaller un OS et que la personne ne se rende compte de rien ? Ben non, mais c'est qu'une série, tu sais...



Il n'y a pas que ça. Le gars, McGee, il a bien rebooté sur sa clé, au début de la séquence. Par contre, pour une *install* normale il aurait dû rebooter une deuxième fois à la fin, pour démarrer sur l'OS fraîchement réinstallé. Et là il n'a rien fait, il a juste débranché la clé USB et il est parti en laissant le système tourner.

Une demo ? Un devs ?

Si vous souhaitez nous présenter vos coordonnées professionnelles, nous vous contacterons rapidement !

- Effectivement, mais pour le public moyen ça passe inaperçu ce genre de détail.

- Peut-être. Mais nous on n'est pas le public moyen. Et à bien y réfléchir, je crois qu'on pourrait très bien programmer un système d'install qui suive ce comportement. C'est ça que je suis en train de tester.

- Ah. Et tu as aussi quelque chose pour expliquer que la réinstallation d'OS passe complètement inaperçue ?

- Ben ouais. Par exemple l'OS du gars pourrait être un partage NFS (*Network File System*). Si c'est le cas, le fait de booter temporairement sur une clé USB en local ne change rien aux données, puisqu'elles sont hébergées ailleurs.

- Du NFS ? Ah ouais pourquoi pas. Et c'est vrai que les gars sont censés bosser dans une agence gouvernementale avec pas mal de personnel... Donc il est tout à fait possible qu'il y ait un système de ce genre pour centraliser les données et le boulot d'administration qui va avec.

- Tout à fait.

- Mais si c'est un partage NFS et que rien n'est modifié sur l'OS au final, comment tu expliques que le système soit trafiqué ?

- Eh bien euh, il y a deux solutions. Le plus simple serait que McGee bidouille directement des trucs sur le serveur NFS distant. Mais dans ce cas, il aurait pu le faire depuis son bureau tranquille, plutôt que de monter toute cette opération pour s'introduire dans le bureau de Devon. Le souci à mon avis, c'est que le serveur central est sans doute hautement sécurisé et surveillé : si McGee tente de s'y introduire et se fait repérer, les complices de l'agent Devon pourraient en avoir connaissance. À l'inverse, le poste local de Devon est sûrement beaucoup plus vulnérable.

Powered by Sensy (<https://sensy.link/019cf09e-334f-70ad-a07f-494f31dd8679-323ee>)

- Je vois. Mais tu n'as pas répondu à ma question...

- Et bien, si on peut bidouiller la procédure de boot en local, on peut par exemple empiler le partage NFS avec une autre couche de *filesystem* qui contiendrait les hacks en question. Je pourrais te simuler ça sur mon portable. Mais si tu n'as pas le background ça va te paraître un peu nébuleux...

3. Les étapes de démarrage d'un OS

Peter se met visiblement à réfléchir sur la façon dont il va pouvoir m'expliquer la chose. Puis il reprend :

- As-tu une vision claire des grandes étapes de démarrage d'un OS **GNU/Linux** ? [1]

- Oui je crois. Quand on met la machine sous tension, le *firmware* s'exécute (BIOS ou UEFI sur un PC), il passe la main au *bootloader*, qui lance le noyau Linux. Celui-ci s'initialise, puis monte la racine du système de fichiers et lance le premier processus (ce qu'on appelle un « système d'init »). Ce premier processus est l'ancêtre de tous ceux qui seront créés par la suite.

Une demo ? Un dev ? C'est à peu près ça, mais ce que tu me décris, c'est le cas le plus simple. En réalité on ne peut pas gérer tous les cas de cette manière. Par exemple, imagine le cas suivant : ton système de fichiers est un *btrfs* avec du RAID logiciel sur 2 disques en dessous. Le *bootloader* est déjà censé trouver le fichier du noyau Linux (et vos coordonnés professionnelles nous ou quelque chose du genre) pour pouvoir le lancer. S'il y arrive dans ce genre d'environnement, il est balaise. **Grub2** y arriverait peut-être, mais je ne suis pas sûr. C'est pour ça qu'on place parfois `/boot` sur une autre partition, dans un environnement moins complexe (par exemple une partition physique en FAT). Ensuite, si on admet que le *bootloader* a pu lancer le noyau, celui-ci doit par contre forcément monter le système de fichier racine. Il y a alors 2 approches. La première est de considérer que le noyau doit savoir gérer ce genre de cas complexe. On implémente donc toutes les technologies concernées (ici *btrfs* et RAID) dans le noyau. Mais ensuite, comment aller chercher un module `/lib/modules/.../btrfs.ko` sur une partition en *btrfs* ?? Il faut donc compiler ces différentes technos *en dur* dans le noyau, et non pas sous la forme de modules. Et l'éditeur de la distribution fournissant en général le même noyau pour tous, il faut aussi inclure tout ce que les autres utilisent, peut-être du NFS, du XFS, du je-ne-sais-quoi et gérer toutes les combinaisons possibles des différentes technos. Tu vois le souci. Donc, depuis un bout de temps, on préfère une autre approche, en tout cas dans les distributions grand public. Cette deuxième approche consiste à intercaler une étape supplémentaire entre le noyau et le système d'init. As-tu remarqué les fichiers `initrd.img-<version>` dans `/boot` ?

- Euh oui... Cela concerne donc l'« étape supplémentaire » dont tu parles ?

- Oui. On appelle ça un *initramfs*. En fait il s'agit d'une archive compressée au format `cpio`^[2] contenant une arborescence de fichiers. Regarde :

```
$ mkdir /tmp/initramfs
$ cd /tmp/initramfs
$ zcat /boot/initrd.img-4.4.0-36-generic | cpio -id
166243 blocs
$ ls
bin  conf  etc  init  lib  run  sbin  scripts  usr  var
```

Powered by Sellsy (<https://sellsy.link/019cf69e-334f-70ad-a07f-494f31dd8679-323ee>)

Tu vois, le contenu de cette archive ressemble à un mini-système, avec une arborescence classique. Et ce mini-système à beau être mini, il sait faire des choses. Par exemple :

```
$ find . -name btrfs.ko
lib/modules/4.4.0-36-generic/kernel/fs/btrfs/btrfs.ko
```

Ce mini-système contient, entre autres, le module noyau nécessaire pour gérer un système de fichier *btrfs*.

- Intéressant... Et donc, tu dis que ce mini-système s'intercale entre le noyau et le système d'init ?

- Oui, grosso modo. Voilà comment ça se passe. En plus du noyau, le *bootloader* charge en RAM cette archive compressée, à une adresse donnée. Et quand il lance le noyau, il lui passe cette adresse via un registre CPU. Le noyau est donc capable de décompresser l'archive et se retrouve avec un système de fichiers stocké en RAM. Au final, après son initialisation, c'est à ce mini-système que le noyau va passer la main, en exécutant le fichier `/init`.



Une demo ? Un dev ?
le vois. Et donc si on a du *btrfs* et du RAID logiciel, ce script pourra charger les modules correspondants. Il sera alors capable de trouver notre système de fichiers final, le monter et exécuter le système d'init. C'est bien ça ?

N'hésitez pas à laisser vos coordonnées professionnelles, nous vous contacterons rapidement !

- Oui tout à fait.

- ~~Coordonnées~~ Dans ce cas, il y a un truc qui me chiffonne... Si j'ai bien compris, on a juste déplacé la complexité depuis le noyau linux vers cet *initramfs*, non ? Si c'est l'*initramfs* qui doit savoir gérer toutes les technos et toutes les combinaisons possibles dont tu parlais tout à l'heure, alors le noyau est effectivement plus léger, mais cet *initramfs* doit être énorme et super complexe !

- Non, pas du tout. Premièrement, il est beaucoup plus facile d'implémenter quelque chose en espace utilisateur plutôt que dans le noyau, donc la complexité est largement moindre. Et en plus, dans les distributions modernes, l'*initramfs* est automatiquement adapté à ton environnement. Par exemple, si tu installes le paquet *linux* sur une **Ubuntu**, l'archive compressée est automatiquement régénérée (via un script qui s'appelle *update-initramfs*) pour que l'*initramfs* puisse gérer cette nouvelle fonctionnalité. Au final, la taille et la complexité de l'*initramfs* sont donc relativement maîtrisées.

- Je vois.

- Dernière chose à ce sujet : tu as vu que le mini-système à beau être mini, il présente une arborescence très classique. L'avantage étant que, pour le noyau, démarrer le système final ou démarrer le mini-système, c'est la même chose. Et on pourrait éventuellement aller plus loin, enchaîner tour à tour plusieurs mini-systèmes de ce type avant d'arriver au système final...

- OK... Je crois que je commence à voir où tu veux en venir, et le lien avec notre épisode de série US. Tu penses qu'en rajoutant une étape de ce type dans le déroulement du boot, on pourrait introduire les hacks nécessaires ?

- Humm... non, pas exactement. Mais c'était bien tenté. En fait, l'*initramfs* de l'agent Devon est chargé de deux choses : d'abord, il doit monter le système de fichiers final, c'est-à-dire le partage NFS, et ensuite lancer le système d'init de cet OS distant. Or, pour introduire les hacks tels que je les imagine, on va devoir bidouiller quelque chose entre ces 2 actions. Donc a priori la seule solution pour ajouter du code à cet endroit-là est de modifier l'*initramfs* existant.

4. De l'*initramfs* au système d'init final

Bon, je te montre.

Pour un test plus réaliste, il me faudrait 2 machines, mais là je n'ai que mon portable. On va donc simuler à la fois le serveur NFS central et la machine locale de l'agent Devon. Commençons par le serveur NFS central :

```
$ su
# apt-get install nfs-kernel-server
# cd /tmp
# debootstrap --variant=minbase xenial os-devon
# ls os-devon/
```



```
bin boot dev ... tmp usr var
# echo "/tmp/os-devon *(rw)" >>/etc/exports
# exportfs -a
```

N'hésitez pas à laisser vos coordonnées professionnelles, nous vous contacterons rapidement !

Voilà, on a préparé un OS **Debian** dans `/tmp/os-devon` et on l'a rendu accessible par NFS.

Coordonnées
Passons à la partie « machine locale ». On considère donc à présent qu'on est sur le PC de l'agent Devon. Pour y croire un peu plus, on peut ajouter une ligne dans `/etc/hosts` :

```
# echo '127.0.0.1 nfs-norfolk' >> /etc/hosts
```

Maintenant, imagine que ce PC est en train de démarrer. Le noyau est chargé, il a lancé le script `/init` de l'*initramfs*. Celui-ci doit alors monter le partage NFS :

```
# mkdir -p mnt/os-devon
# mount -t nfs nfs-norfolk:/tmp/os-devon mnt/os-devon
# ls mnt/os-devon/
bin boot dev ... tmp usr var
```

À ce stade, dans son fonctionnement normal, l'*initramfs* est censé passer la main à l'OS hébergé à distance, en faisant quelque chose comme :

```
# cd mnt/os-devon
# exec chroot . sbin/init
```

- ?? Qu'est-ce que ça fait, ce `exec chroot` machin ??

- Analysons la chose en commençant par la fin. Pour démarrer un OS, il faut lancer le premier processus, celui qui aura le PID **1**, et ce processus sera chargé de lancer tous les autres. C'est ce qu'on appelle le « système d'init », et d'après ce que tu m'as dit tout à l'heure, tu connais déjà. Comme tu peux t'en douter, ce système d'init est démarré en exécutant `/sbin/init`. En fait, comme il y a plusieurs implémentations possibles, il s'agit d'un lien symbolique :

Powered bySellsy (<https://sellsy.link/019cf69e-334f-70ad-a07f-494f31dd8679-323ee>)

```
# ls -l /sbin/init
lrwxrwxrwx 1 root root 20 sept. 29 04:01 /sbin/init -> /lib/systemd/systemd
```

Tu vois, sur mon système c'est **systemd** qui se cache derrière.

- OK... Tout s'éclaire.

- Revenons à notre cas d'étude, avec le système distant monté par NFS. Le problème à ce stade, c'est qu'on n'est pas dans un environnement adéquat pour lancer un système d'init. En effet, les chemins des fichiers ne sont pas ceux attendus : à la place de `/sbin/init`, on a `[...] /os-devon/sbin/init`, à la place de `/bin/sh`, on a `[...] /os-devon/bin/sh`, le répertoire `/lib` est en réalité `[...] /os-devon/lib`, etc. Il est clair que le programme d'init ne va pas fonctionner correctement dans ces conditions ! En fait, ce qu'il faudrait, c'est *déplacer la racine du système de fichiers*, pour que le symbole `/` pointe dorénavant vers le répertoire `[...] /os-devon`.

- Pourquoi pas... si je comprends bien, ça reviendrait à changer de référentiel sur un graphique... Mais c'est possible un truc pareil ?



- Ben oui, t'es mauvais en anglais ou quoi ? Dans l'instruction `chroot`, il y a *root* ce qui signifie *racine*, et *ch* c'est pour *change* !

- Ah oui, j'aurais pu deviner :)

N'hésitez pas à laisser vos coordonnées professionnelles, nous vous contacterons rapidement !

- C'est clair. Tu ferais bien de reprendre un café. Donc l'instruction `chroot` permet de changer la racine du système de fichiers. Le premier paramètre correspond à la nouvelle racine, ici le répertoire courant. Le deuxième correspond à l'exécutable à lancer dans ce nouvel « environnement ».

- Ça va changer la racine de tous les processus lancés à partir de maintenant ?

- Heu... C'est un peu plus subtil que ça. En fait, la notion de racine du système de fichiers, c'est une notion propre à chaque processus. Mais quand un processus donné a besoin de créer un fils, il fait un `appel système fork()`, et le noyau duplique ce processus. Cet « attribut » se retrouve donc dupliqué, et le processus fils se retrouve avec la même racine que son père.

- Je vois... Et comme c'est le processus d'init qu'on manipule, s'agissant du premier processus, tous ceux créés par la suite hériteront de proche en proche de ce même attribut. C'est bien ça ?

- Oui. Je vois que tu as bien compris. Donc passons au `exec`. Tu sais à quoi ça sert ?

- Non, pas plus.

- En fait, quand tu lances une commande, dans un script shell, cela provoque la création d'un processus fils chargé d'exécuter cette commande. Le père se met alors en attente, et quand la commande est terminée (cela signifie donc que le fils s'est arrêté) il reprend la main pour poursuivre le script. Si tu préfixes la commande par `exec`, ce comportement est modifié. Aucun processus fils n'est créé, et c'est le père qui prend en charge la commande. Si on va au bout du raisonnement, on s'aperçoit que `exec` est toujours la dernière commande exécutée par le script. En effet, quand le père prend en charge la commande, il n'y a plus personne pour s'occuper du script, donc personne pour reprendre la main après le `exec`. C'est définitif.

- C'est bien bizarre comme instruction ! Et à quoi ça sert ??

- Et bien dans notre cas, le fils serait le système d'init. Ce processus étant censé vivre jusqu'à l'arrêt de la machine, il n'est pas utile de garder un processus père en attente pour rien.

- Donc c'est juste une question d'économie de ressources ?

- Non, il y a une autre raison, plus importante d'ailleurs. Pour bien comprendre, il faut se mettre à la place du noyau. Pour lui, un `initramfs` ou un système final, c'est la même chose : le script `/init` de l'`initramfs`, c'est déjà un « système d'init ». Et comme c'est son premier bébé, il le lance avec le PID 1. Maintenant, à la fin de ce script, on doit passer la main au système d'init final. Si on préfixe avec `exec`, comme on reste sur le processus père, on reste avec ce même PID 1. Sinon, s'agissant d'un fils, le système d'init final aurait un PID différent de 1.

- Et alors ? Où est le problème ? Le fait d'avoir un PID différent de 1 pourrait déstabiliser le système d'init ??

- Oui, aussi étonnant que cela puisse te paraître... Cela vient du fait que la commande `/sbin/init` est utilisée pour 2 choses différentes. Soit elle assure son rôle de système d'init, soit elle contrôle ce système d'init. As-tu déjà tapé `init 6` pour rebooter ta machine ?

Euh oui. C'est le même exécutable `/sbin/init` qui est appelé dans ce cas ?

- Oui, bien sûr ! Et tu vois bien que dans ce cas on ne veut pas lancer une autre instance du système d'init, on veut juste le contrôler. On a donc 2 fonctionnalités complètement différentes à gérer dans ce même exécutable. Et est-ce que tu devines comment cet exécutable détermine la fonctionnalité à assurer ?

- Ne me dis pas qu'il vérifie son numéro de PID ??

- Si !! Je suis d'accord avec toi que c'est un peu étrange et pas très intuitif, mais j'imagine qu'on a gardé ce fonctionnement pour des raisons historiques... Toujours est-il que, de ce fait, l'emploi du `exec` pour conserver le PID 1 entre l'*initramfs* et le système d'init final est primordial. Voilà, tu sais tout sur la commande `exec chroot . /sbin/init` que j'ai introduite tout à l'heure.

- Merci pour les explications. Si je récapitule, cette commande permet donc de lancer l'OS sur le partage NFS, à la fin de l'*initramfs*. C'est bien ça ?

- Oui. En tout cas on peut imaginer que le système de l'agent Devon fonctionnait de cette manière, avant qu'il soit piraté.

5. Une union de filesystems

- Je vois... Et donc, comment est-ce que tu comptais modifier ce fonctionnement ?

- Et bien, juste avant de passer la main au système d'init final, on pourrait monter une union avec nos hacks stockés sur la clé USB, via un montage avec le *filesystem* adéquat.

- Une union ? C'est-à-dire ?

- Le principe d'une union, c'est grosso-modo de pouvoir combiner plusieurs systèmes de fichiers au niveau d'un seul point de montage. Je te montre.

Powered bySellsy (<https://sellsy.link/019cf69e-334f-70ad-a07f-494f31dd8679-323ee>)

```
# cd /tmp
# mkdir -p hacks unionwork mnt/union
# modprobe overlay
# mount -t overlay -o upperdir=hacks,lowerdir=mnt/os-devon,workdir=unionwork none mnt/union
```

Voilà, notre union est montée dans `mnt/union`. J'ai utilisé le système de fichiers `overlay` [3] pour la construire. Comme tu le vois dans les options, on a construit l'union des répertoires `hacks` et `mnt/os-devon`. Cela signifie que tous les fichiers et répertoires que l'on trouve dans l'un ou l'autre de ces répertoires seront visibles dans notre point de montage `mnt/union`. Bien évidemment, comme `hacks` est vide pour l'instant, dans `mnt/union` on retrouve ni plus ni moins que le contenu de `mnt/os-devon` :

```
# ls mnt/union
bin boot dev ... tmp usr var
```

Mais si on ajoute des choses dans `hacks`...



Une demo ? Un devis ?
touch hacks/mytest

ls mnt/union

bin boot dev ... mytest ... tmp usr var

N'hésitez pas à laisser vos coordonnées professionnelles, nous vous contacterons rapidement !

... les modifs apparaissent dans l'union.

Coordonnées

- Et en cas de conflit ? Si on trouve le même chemin de fichier dans `hacks` et `mnt/os-devon` ?

- J'allais y venir. La priorité est sur la « couche supérieure » (l'option de montage `upperdir`) donc `hacks`. Regarde :

```
# mkdir hacks/bin
```

```
# echo hacked >> hacks/bin/bash
```

```
# cat mnt/union/bin/bash
```

```
hacked
```

- OK... Par contre il y a encore des trucs pas clairs dans ta ligne de commandes `mount`... L'option de montage `workdir=unionwork`, tu n'en as pas parlé... Et le `none`, avant le point de montage, à quoi ça sert ?

- Oh ça ce n'est pas très important. Le `workdir` c'est, comme son nom l'indique, un répertoire de travail pour le montage `overlay`. Si je me souviens bien, il en a besoin pour implémenter certaines opérations de manière atomique. Et le `none` est là uniquement pour respecter la syntaxe du `mount`. Parce qu'historiquement, on montait toujours un *device* sur un répertoire. Alors que maintenant, il existe plusieurs systèmes de fichiers de ce genre pour lesquels on n'a pas de *device* associé. Donc on met juste un mot quelconque à cet emplacement pour respecter la syntaxe.

Bon, je ne vais pas m'étendre davantage sur ce sujet, mais sache que ça marche plutôt pas mal, ce genre de *filesystem*. Sur l'union, tu peux modifier les fichiers à loisir, les modifications seront reportées sur la couche `upperdir`, donc `hacks`. La couche `lowerdir` (ici `mnt/os-devon`) restera intacte, quoi que tu fasses. Pour des cas pathologiques (suppression dans l'union d'un fichier de `mnt/os-devon`), le *filesystem* crée même des fichiers spéciaux dans la branche `hacks`. Tu pourras essayer si ça t'intéresse.

En tout cas, tu vois ce que j'imaginai : plutôt que de lancer le système d'init final sur `mnt/os-devon`, on le lance sur l'union, ça nous permet d'inclure toute une couche de hacks pour pirater la machine.

- Oui je vois bien. Par contre, d'où tu sors cette techno de derrière les fagots ? Je veux dire, OK ça a l'air sympa et tout, mais à quoi ça peut bien servir ? C'est juste pour hacker des OS au démarrage ?? Il y a un gars qui s'est tapé des jours et des jours de programmation noyau pour ça ??

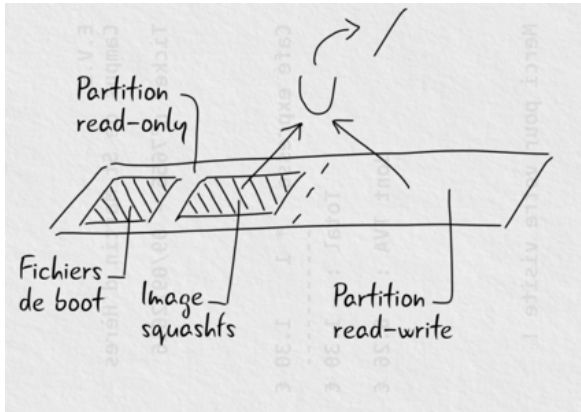
- Non, bien sûr que non ! En fait, ça existe depuis un bout de temps ! On a introduit ça pour les live-CDs : en faisant une union, l'OS pouvait combiner tous les fichiers en lecture seule sur le CD avec une zone de stockage en RAM, donc accessible en écriture. Quand on modifiait ou supprimait un fichier sur l'union, la modif était enregistrée côté RAM. Ainsi, on avait vraiment l'impression d'avoir booté un média en lecture/écriture...

- OK, je vois le principe... Mais j' imagine que la techno a presque disparu avec l'omniprésence des clés USB. Plus besoin d'union sur un média accessible en lecture/écriture...

Une demo ? Un dev ? Détronne-toi. On a gardé grosso modo le même principe. La clé USB a beau être réinscriptible, on l'utilise en grande partie en lecture seule. Il y a deux raisons à cela, je crois. La première, c'est de fabriquer des images compatibles à la fois avec les CDs et les clés USB. La deuxième, c'est que les systèmes de fichiers en lecture seule peuvent être beaucoup mieux compressés.

Peter commença alors à griffonner un schéma au dos de la note de son café (voir figure 1).

Coordonnées



(https://connect.ed-diamond.com/sites/default/files/articles/gnu-linux-magazine/glmf-206/83066/structure_liveusb1.png)

Fig. 1 : Structure usuelle d'un LiveUSB (au dos de la note de chez EVE).

Puis il reprit :

- Tu vois, on utilise un système de fichiers compressé, `squashfs` en général, pour stocker autant de fichiers que possible sur la clé USB. Et on peut faire une union avec un autre système de fichiers, stocké sur une petite partition qu'on utilisera en lecture/écriture. Ainsi, contrairement au CD qui enregistre en RAM, les modifs sont cette fois-ci stockées de manière pérenne sur cette partition, et donc tu les retrouveras si tu redémarres l'OS.

- Je vois. N'empêche que la moindre clé USB fait 8 Gio de nos jours... Est-il nécessaire de garder une structure aussi complexe ? Et concernant la compatibilité avec les CDs, l'argument me paraît discutable... Qui aujourd'hui grave un CD pour booter un OS ?

- Tu as sans doute raison. Surtout que cette complexité a des inconvénients, par exemple on ne peut pas mettre à jour le noyau ou le *bootloader* sur ce genre de LiveUSB. Mais bon. Il faut du temps pour que les choses changent.

6. Migration de disque à disque

Avant même la fin de cette phrase, Peter était visiblement reparti dans ses pensées. Il lançait de temps en temps une phrase explicative, mais je n'étais pas sûr qu'elles m'étaient destinées. J'avais plutôt l'impression qu'il réfléchissait tout haut. À un moment, il finit quand même par se tourner

vers moi.
Une demo ? Un devis ?

- Tu vois ce que je veux dire ?

- N'hésitez pas à laisser vos coordonnées professionnelles, nous vous contacterons rapidement !
Euh... à vrai dire...

- Reprenons. Tu es d'accord qu'en modifiant l'*initramfs*, on peut introduire ce système d'union et hacker l'OS sans que l'agent s'en aperçoive ?

- Oui, ça me semble plausible.

- OK, donc passons à l'autre question. Dans l'épisode, l'ordi de l'agent Devon démarre sur la clé USB, jusqu'à afficher un bureau, puis une barre de progression. À la fin de cette barre de progression, l'agent McGee débranche la clé USB, à chaud, et laisse tourner l'OS. Ça c'est inhabituel.

- C'est vrai. On dirait un genre d'install sans redémarrage.

- Oui, grosso modo. En fait, rien ne nous empêche de rajouter encore quelques instructions dans notre script *initramfs*, de façon à copier l'archive *initramfs* modifiée sur la machine locale. Car sinon, au premier reboot sans la clé USB, l'OS retrouvera son comportement initial.

- Oui c'est clair.

- Le souci, c'est que si l'OS tourne effectivement sur une union, alors on a la majeure partie de l'OS sur le serveur NFS distant, mais on a aussi un répertoire `hacks` en local, sur la clé USB. Il faudra donc également copier ce répertoire sur la machine locale, pour que notre bidouille résiste au prochain reboot. Et a priori, ce répertoire `hacks` doit bien peser quelques centaines de mégas, au bas mot.

- Tant que ça ?

- Écoute, d'après l'épisode, dans les programmes hackés il y a au moins le client mail, et ce n'était visiblement pas un programme très léger... En plus, j'imagine que McGee n'a pas pris de risques et a compilé tous ces programmes en `static`. Sinon, il prend le risque d'incompatibilités avec les bibliothèques partagées stockées côté NFS, et dont la version n'est peut-être pas celle attendue...

- Je vois... Donc oui pourquoi pas, quelques centaines de mégas, au moins. Ça expliquerait la barre de progression.

- Oui. Mais cette idée de copie au moment de l'*initramfs* ne me semble pas du tout cohérente avec ce qu'on voit dans l'épisode. Parce que la barre de progression apparaît vraiment tard, après l'affichage du bureau. Et il y a même autre chose qui me chiffonne. On a vu qu'avec un système d'union, les modifs sont stockées dans la « couche supérieure ». Cela veut dire que si l'agent Devon crée un nouveau document **LibreOffice** par exemple, il sera stocké quelque part dans `hacks`. S'il reçoit un mail, c'est pareil, il sera stocké dans `hacks`. Donc au final, pas de souci pour copier la version initiale de `hacks` au boot, mais ensuite ? L'OS va continuer à faire des modifs dans le répertoire `hacks` de la clé USB, pas sur la copie locale ! Et si on retire la clé USB, ça va clairement planter !

- C'est clair... Mais attends ! Je crois que ça me rappelle un truc ton histoire... un outil justement capable de « migrer à chaud » l'OS depuis la clé USB vers le disque local... comment il s'appelle déjà ?... `debootstick`, oui c'est ça ! On pourrait regarder comment il fonctionne !

- `debootstick` ?



Une demo ? Un devis ?
- Oui, c'est un outil qui permet de créer des OS bootables. L'auteur a écrit un article dans *GNU/Linux Magazine* récemment [4]... Ce n'était pas le cœur de l'article, mais il a parlé de `debootstrap` dans l'intro. Ça avait l'air sympa, alors je l'ai testé pour voir. Et il me semble bien avoir vu une option de ce type dans la page de manuel.
N'hésitez pas à laisser des coordonnées professionnelles, nous vous contacterons rapidement !

- C'est *open source* ?
Coordonnées

- Je crois oui...

Après une petite recherche **Google**, Peter s'était mis à explorer le dépôt de `debootstrap` sur **GitHub**[5]. Et avec une rapidité qui m'étonnera toujours, il avait déjà trouvé le bout de code qui l'intéressait. Je vais finir par croire que les neurones de ce gars sont organisés en table de hachage...

- Mais oui ! Évidemment !! C'est ça la solution, il suffit de jouer sur les volumes physiques LVM !!

- Si tu le dis. Et c'est quoi LVM ?

- « Logical Volume Management ». LVM, en gros, c'est une abstraction qui te permet de manipuler tes disques avec beaucoup de souplesse. Normalement, ce que tu manipules, c'est des partitions. Et manipuler des partitions c'est un peu comme demander un peu de fantaisie à un comptable allemand rigoureux : tu es quand même très limité. Prenons par exemple le cas classique où tu manques d'espace disque sur une de tes partitions. Si, par un heureux hasard, cette partition est suivie d'un espace libre non utilisé, alors tu peux l'étendre assez facilement, mais c'est un cas super rare. Alors qu'avec LVM, on travaille sur des « volumes logiques » plutôt que sur des partitions. Et un volume logique ne correspond pas forcément à un espace contigu sur le disque, on est donc beaucoup plus souple. On peut même étendre un volume logique sur plusieurs disques.

- Je vois... Et donc, c'est à quel endroit dans ce code que `debootstrap` utilise LVM ?

- Attends, il faut que je t'explique un peu plus de trucs sur LVM, et sa structure en « couches ». Sur la couche la plus basse, on a la notion de « volume physique ». Un volume physique, c'est un « espace libre sur un disque ». Ça peut donc être une partition du disque, ou bien un disque complet. Ensuite on a une couche intermédiaire qui nous permet de créer des « groupes de volumes physiques ». Et enfin, dans chaque groupe de volumes, on peut créer 1 à n « volumes logiques », volumes qu'on pourra utiliser comme de bêtes partitions (donc en général mettre un *filesystem* dessus).

- Ah ouais, quand même. Je me demande toujours quel genre de cerveau peut nous pondre une techno de ce genre...

- Réfléchis, c'est relativement logique... Imagine que tu doives stocker beaucoup de données sur un serveur. Tu comptes mettre l'OS sur un SSD et pour les données tu as 4 disques de 4 To chacun. Eh bien, pour faire simple, tu peux formater chaque disque comme un « volume physique » LVM. Ensuite tu mets le SSD tout seul dans un groupe de volumes que tu appelles `VG_OS`, et les 4 autres disques dans un groupe de volumes `VG_DATA`. Ensuite, tu crées tes « volumes logiques » : par exemple 3 volumes `LV_ROOT`, `LV_BOOT` et `LV_SWAP` dans `VG_OS`, et un seul volume `LV_DATA` dans `VG_DATA`. Tu formates tous ces volumes avec le *filesystem* qui va bien, et c'est parti. Et quand tu auras épuisé tes 16 To de disque avec ta grosse appli, tu me remercieras. Parce que tu auras juste à brancher 1 disque supplémentaire dans ta super baie, l'ajouter à `VG_DATA` et tu pourras alors tranquillement étendre `LV_DATA` (et ensuite le *filesystem*) sur l'espace libre.



- Effectivement sur ce genre d'exemple ça paraît censé.

Ah, tu vois. Et ce n'est pas tout. Imagine, 6 mois plus tard tu reçois une alerte remontée par ton système de monitoring. Le contrôleur de disques t'annonce que ton deuxième disque de données (disons `/dev/sdc`) est passé en « predictive failure » : il va probablement bientôt rendre l'âme. Dans ce cas, avant que cela n'arrive, voilà ce que tu dois faire :

1. brancher un nouveau disque dans la baie pour s'assurer qu'on a de l'espace libre. Admettons que ce nouveau disque s'appelle `/dev/sdf` ;
2. formater `/dev/sdf` lui aussi comme un volume physique LVM ;
3. ajouter `/dev/sdf` au groupe `VG_DATA` ;
4. indiquer à LVM de ne plus allouer de nouveaux blocs sur `/dev/sdc` (le disque en « predictive failure »). LVM sera donc obligé d'utiliser les autres disques de `VG_DATA` pour stocker les nouvelles données ;
5. indiquer à LVM de libérer les blocs alloués sur `/dev/sdc` : cette instruction va provoquer la migration des blocs alloués vers les autres disques de `VG_DATA` ;
6. une fois la migration terminée, enlever `/dev/sdc` du groupe `VG_DATA` ;
7. débrancher `/dev/sdc` de la baie.

Et voilà. Tu as résolu ton souci à chaud, sans redémarrage d'OS, et sans le moindre impact sur tes applications, même celles qui lisaient ou écrivaient dans `LV_DATA` pendant nos manipulations.

- Impressionnant ! Mais on ne s'éloigne pas un peu de notre propos, du code de `debootstrap` et de nos soucis de boot ?

- Non, pas du tout ! Regarde, ici, dans les sources de `debootstrap` [6] :

```
yes | pvcreate -ff ${TARGET}3
vgextend $LVM_VG ${TARGET}3
pvchange -x n ${ORIGIN}3
pvmove -i 1 ${ORIGIN}3 | while read pv action percent
do
    echo REFRESHING $pv $percent
done
vgreduce $LVM_VG ${ORIGIN}3
```

D'après ce que j'ai compris, au moment où on arrive sur ce code, `${ORIGIN}` c'est la clé USB et `${TARGET}` c'est le disque interne. D'autre part, apparemment, le *filesystem* de l'OS embarqué sur la clé est en réalité hébergé sur un volume LVM. Ce volume repose sur le groupe de volumes `$LVM_VG`, qui ne contient au départ qu'un seul volume physique, `${ORIGIN}3`, donc la troisième partition de la clé USB. Le but de ce bout de code est alors de forcer LVM à migrer les données depuis `${ORIGIN}3` vers `${TARGET}3`, donc depuis la clé USB vers le disque interne.

- Un peu comme tu expliquais juste avant, avec la migration des données d'un disque défaillant vers les autres disques ?

- Oui tout à fait. En réalité, les 5 instructions LVM que tu vois dans ce bout de code, c'est l'exacte traduction des étapes 2 à 6 que j'ai listées.

- D'accord...



Et ce qui est puissant avec cette technique, c'est qu'on peut migrer à n'importe quel moment de la vie de l'OS. Donc l'histoire de la barre de progression qui s'affiche bien tard, alors que le bureau est déjà lancé, ça paraît tout à fait plausible avec cette technique.

N'hésitez pas à laisser vos coordonnées professionnelles, nous vous contacterons rapidement !
Et accessoirement, tu vois aussi dans ce code qu'il n'est pas difficile de récupérer un pourcentage d'avancement, pour par exemple afficher cette barre de progression !

7. Le déroulé final

Il avait failli renverser mon reste de café en voulant mimer la barre de progression. Il s'arrêta à nouveau pour réfléchir quelques secondes, puis reprit :

- A priori on peut valider tout ce qui se passe dans l'épisode. Tu es d'accord ?
- Oui, je crois aussi...
- Alors vas-y, explique-moi comment boote la clé USB « magique » de McGee.
- Eh bien, au départ, tout à fait normalement... Jusqu'au montage NFS. Ensuite, on peut penser qu'elle monte une union avec des données stockées sur la clé (ce qu'on a appelé le répertoire `hacks`). Elle lance le système d'init final sur cette union. Et elle commence aussi à migrer tous les éléments de cette clé USB pirate vers le disque interne de la machine, pour que ce système soit conservé au prochain reboot. En utilisant LVM, a priori.
- Oui, c'est ça. Dernière précision : quand on passe d'un processus d'init (*initramfs*) au suivant (init final), la bienséance voudrait qu'on passe la main dans un état le plus propre possible. En particulier, on évite de garder en vie des processus créés dans l'*initramfs*. Mais rien ne nous empêche de le faire en réalité. Donc, juste avant de passer la main, notre *initramfs* peut très bien lancer le processus de migration en arrière-plan.
- Bon. Sur ce, je rentre chez moi alors, il est temps.
- OK, merci pour l'info concernant `debootstrap`. Je regarderai ce soft plus en détail.
- De rien.

Je n'ai rien dit sur le coup, mais j'estimais plutôt que c'était moi qui avais appris tout un tas de trucs, en une demi-heure de temps !

Conclusion

À présent, si je regarde un film ou une série, il suffit qu'un personnage fasse quelque chose d'inhabituel avec un ordi et je ne peux m'empêcher de me poser la question : « Impossible ou juste invraisemblable ? »...

Souvent on reste dans le classique, avec par exemple le logiciel de reconnaissance faciale d'une lenteur abominable parce qu'il affiche tous les visages qu'il analyse... Mais il y a pas mal d'autres scènes qui posent davantage question, et mériteraient clairement un petit test de faisabilité. Et là on commence à y réfléchir, jusqu'à perdre le fil de l'histoire. Tout ça à cause de « Peter Le Testeur ». Il m'a passé le virus. J'espère donc que je ne vous ai pas contaminé à votre tour, chers lecteurs ;)

Merci à Henry et Narek pour les relectures !
Une demo ? Un dev ?

Notes et références

Si vous n'avez pas réussi à résoudre vos problèmes professionnels, nous vous contacterons rapidement !

[1] Dans cet article, j'insiste principalement sur le « pourquoi » des différents éléments d'un OS. Une approche alternative et complémentaire pour bien comprendre est de consulter l'article de F. ENDRES, « *Live-System from Scratch* », que l'on peut trouver dans *GNU/Linux Magazine* n°202 (mars 2017, p.54 à 61). L'auteur y explique comment intégrer les différents éléments d'un OS pour créer une clé USB bootable.

[2] L'archiveur `cpio` est un vieux concurrent de `tar`, qui à ma connaissance n'est plus utilisé, sauf dans ce cas précis des archives *initramfs*.

[3] Le système de fichiers `overlay` (ou `overlayfs`) est disponible dans le noyau *mainline* depuis la version 3.18, on le trouve donc dans la plupart des distributions modernes.

[4] DUBLÉ É., « *Constructions with en langage bash* », *GNU/Linux Magazine* n°204, mai 2017, p.70 à 78.

[5] Page GitHub de `debootstick` : <https://github.com/drakkar-lig/debootstick>
(<https://github.com/drakkar-lig/debootstick>)

[6] À l'heure où ces lignes sont écrites, il s'agit de `scripts/live/init/migrate-to-disk.sh`, lignes 62 à 69. L'URL <http://tiny.cc/debootstick-migrate> (<http://tiny.cc/debootstick-migrate>) pointe sur ce passage.

Powered by Sellsy (<https://sellsy.link/019cf69e-334f-70ad-a07f-494f31dd8679-323ee>)

Article rédigé par

Dublé Etienne (/auteur/duble-etienne)

11 articles

DE

(/auteur/duble-etienne)





Une démonstration ?

Par le(s) même(s) auteur(s)

N'hésitez pas à laisser vos coordonnées si vous souhaitez en savoir plus sur cet auteur : [PLUS D'ARTICLES DE CET AUTEUR \(/SEARCH/NODE?AUTHORS%5B0%5D=71685\)](/search/node?AUTHORS%5B0%5D=71685)

Coordonnées



Le cerveau de Peter (/GNU-Linux-Magazine/glmf-219/le-cerveau-de-peter)

Par Dublé Etienne (/auteur/duble-etienne)

Humeur et Critique (/search/node?domains%5B0%5D=72472)

Comment peut-on gérer plusieurs choses à la fois dans un programme informatique ?
Avec plusieurs threads, évidemment ! C'est bien la seule solution, non ?

...

LIRE L'ARTICLE (/GNU-LINUX-MAGAZINE/GLMF-219/LE-CERVEAU-DE-PETER)



(/readlist/toggle_favorites/83840)



(/readlist/get/modal/83840)



Une démo ? Un devis ?



Constructions « with » en langage... bash ! (/GNU-Linux-Magazine/glmf-204/constructions-with-en-langage...-bash)

N'hésitez pas à laisser vos coordonnées professionnelles, nous vous contacterons rapidement !

Par Dublé Etienne (/auteur/duble-etienne)

Système (/search/node?domains%5B0%5D=72451)

Code (/search/node?domains%5B0%5D=72464)

Dis-moi petit syntax checker, qu'as-tu à me dire sur mon code source ? Comment ça, tu me rends la main tout de suite ? Tu n'as rien trouvé ?? Pas le moindre petit warning ??? Allez, ne sois pas timide, je t'ai mis l'option ultra-verbose je te signale !! T'es pas censé être timide !! Écoute, tu vois bien que je travaille seul là-dessus... si tu ne me dis rien, qui d'autre va me faire la conversation ?? Bon. Tu l'auras voulu. Voyons si tu resteras de marbre quand j'aurai ajouté ...

LIRE L'ARTICLE (/GNU-LINUX-MAGAZINE/GLMF-204/CONSTRUCTIONS-WITH-EN-LANGAGE...-BASH)



(/readlist/toggle_favorites/82984)



(/readlist/get/modal/82984)



Les secrets de fabrication d'une entrée gagnante de l'IOCCC (/GNU-Linux-Magazine/glmf-197/les-secrets-de-fabrication-d-une-entree-gagnante-de-l-ioccc)

Powered bySellsy (https://sellsy.link/019cf69e-334f-70ad-a07f-494f31dd8679-323ee)

Par Dublé Etienne (/auteur/duble-etienne)

Code (/search/node?domains%5B0%5D=72464)

Vous rappelez-vous des articles « Le coin du vieux barbu » où David Odin analysait les entrées gagnantes de l'IOCCC ? Je vous propose de passer de l'autre côté du miroir, afin de découvrir la recette que j'ai employée pour cuisiner mon entrée gagnante de l'IOCCC 2015. ...

LIRE L'ARTICLE (/GNU-LINUX-MAGAZINE/GLMF-197/LES-SECRETS-DE-FABRICATION-D-UNE-ENTREE-GAGNANTE-DE-L-IOCCC)



(/readlist/toggle_favorites/82542)



(/readlist/get/modal/82542)



Une démo ? Un devis ?



La magie des filesystems : 1- Tour d'horizon (/GNU-Linux-Magazine/glmf-168/la-magie-des-filesystems-1-tour-d-horizon)

N'hésitez pas à laisser vos coordonnées professionnelles, nous vous contacterons rapidement !

Coopérateur Double Etienne (/auteur/duble-etienne)

Système (/search/node?domains%5B0%5D=72451)

Le filesystem... Si notre OS préféré a plusieurs modules intéressants, c'est peut-être le plus sympa ! Cette première partie vous propose de le rencontrer. ...

LIRE L'ARTICLE (/GNU-LINUX-MAGAZINE/GLMF-168/LA-MAGIE-DES-FILESYSTEMS-1-TOUR-D-HORIZON)



(/readlist/toggle_favorites/81505)



(/readlist/get/modal/81505)



Les listes de lecture

Python niveau débutant (/liste-de-lecture/python-niveau-debutant-0)

Powered bySellsy (https://sellsy.link/019cf69e-334f-70ad-a07f-494f31dd8679-323ee)

9 article(s) - ajoutée le 01/07/2020

Code (/search/node?domains=72464)

Vous désirez apprendre le langage Python, mais ne savez pas trop par où commencer ? Cette liste de lecture vous permettra de faire vos premiers pas en découvrant l'écosystème de Python et en écrivant de petits scripts.

LIRE LA SUITE (/LISTE-DE-LECTURE/PYTHON-NIVEAU-DEBUTANT-0)

Au pays des algorithmes (/liste-de-lecture/au-pays-des-algorithmes-0)

11 article(s) - ajoutée le 01/07/2020



Une démonstration de [Algo \(/search/node?domains=74474\)](/search/node?domains=74474)

La base de tout programme effectuant une tâche un tant soit peu complexe est un algorithme, une méthode permettant de manipuler les données pour obtenir un résultat attendu. Dans cette liste, vous pourrez découvrir quelques spécimens d'algorithmes.

Coordonnées

LIRE LA SUITE (/LISTE-DE-LECTURE/AU-PAYS-DES-ALGORITHMES-0)

Analyse de données en Python (/liste-de-lecture/analyse-de-donnees-en-python-0)

10 article(s) - ajoutée le 01/07/2020

[Code \(/search/node?domains=72464\)](/search/node?domains=72464)

À quoi bon se targuer de posséder des pétaoctets de données si l'on est incapable d'analyser ces dernières ? Cette liste vous aidera à "faire parler" vos données.

LIRE LA SUITE (/LISTE-DE-LECTURE/ANALYSE-DE-DONNEES-EN-PYTHON-0)



PLUS DE LISTES DE
LECTURE

(/RECHERCHE/LISTES-DE-LECTURE?
DOMAINS%5B74528%5D=74528&DOMAINS%5B74525%5D=74525)

Powered bySellsy (<https://sellsy.link/019cf69e-334f-70ad-a07f-494f31dd8679-323ee>)

Abonnez-vous dès maintenant et profitez de tous les contenus en illimité

**JE DÉCOUVRE LES OFFRES
(/ABONNEMENT/OFFRE-
INTEGRALE)**

Déjà abonné ? [Connectez-vous](#)



Une démo ? Un devis ?

N'hésitez pas à laisser vos coordonnées professionnelles, nous vous contacterons rapidement !

Coordonnées



Appelez-nous (tel:0367100020)

AIDE & CONTACT



NOS PUBLICATIONS



DOMAINES



ABONNEMENTS



À PROPOS



Powered bySellsy (<https://sellsy.link/019cf69e-334f-70ad-a07f-494f31dd8679-323ee>)

